

1) Implement DDA line drawing algorithm to draw Simple line. Divide the screen in four quadrants with center as (0, 0). The line should work for all the slopes

```
#include<GL/glut.h>
#include<iostream>
#include<cmath>
using namespace std;

void init()
{
    glClearColor(1.0,1.0,1.0,1.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-320,320,-240,240);
}

void drawAxes()
{
    glColor3f(0.0,0.0,0.0);
    glBegin(GL_LINES);
    glVertex2d(-320,0);
    glVertex2d(320,0);
    glVertex2d(0,-240);
    glVertex2d(0,240);
    glEnd();
}

void dda(int x1,int y1,int x2,int y2){
    float dx=x2-x1;
    float dy=y2-y1;
    int steps= abs(dx)>abs(dy)?abs(dx):abs(dy);

    float xinc=dx/steps;
    float yinc=dy/steps;

    glColor3f(1.0,0.0,0.0);
    glBegin(GL_POINTS);

    for(int i=0; i<=steps; i++){
        glVertex2d(round(x1),round(y1));
        x1+=xinc;
        y1+=yinc;
    }
    glEnd();
}
```

```

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    int x1,y1, x2, y2;
    cout<<"Enter the x1 y1 x2 y2:";
    cin>>x1>>y1>>x2>>y2;

    dda(x1,y1,x2,y2);
    glFlush();
}

int main(int argc,char**argv)
{
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(640,480);
    glutInitWindowPosition(100,100);
    glutCreateWindow("DDA Line Drawing");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

2) Implement Bresenham line drawing algorithm to draw Dashed line. Divide the screen in four quadrants with center as (0, 0). The line should work for all the slopes

```

#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

// Initialize OpenGL
void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-320, 320, -240, 240); // Coordinate system
}

// Draw X and Y axes

```

```

void drawAxes() {
    glColor3f(0, 0, 0); // Black color
    glBegin(GL_LINES);
    glVertex2i(-320, 0); glVertex2i(320, 0); // X-axis
    glVertex2i(0, -240); glVertex2i(0, 240); // Y-axis
    glEnd();
}

// Bresenham's algorithm with dashes
void drawDashedBresenham(int x1, int y1, int x2, int y2) {
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);
    int sx = x2 > x1 ? 1 : -1;
    int sy = y2 > y1 ? 1 : -1;
    int err = dx - dy;
    int count = 0;

    glColor3f(1.0, 0.0, 0.0); // Red color
    glBegin(GL_POINTS);

    while (true) {
        if (count % 10 < 5) // Draw 5 points, skip 5 (dashed effect)
            glVertex2i(x1, y1);

        if (x1 == x2 && y1 == y2) break;

        int e2 = 2 * err;
        if (e2 > -dy) { err -= dy; x1 += sx; }
        if (e2 < dx) { err += dx; y1 += sy; }

        count++;
    }

    glEnd();
}

// Display function
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    int x1, y1, x2, y2;
    cout << "Enter x1 y1 x2 y2: ";
    cin >> x1 >> y1 >> x2 >> y2;

    drawDashedBresenham(x1, y1, x2, y2);
    glFlush();
}

```

```

}

// Main function
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Dashed Bresenham Line with Axes");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

3) Implement Bresenham line drawing algorithm to draw Solid line. Divide the screen in four quadrants with center as (0, 0). The line should work for all the slopes

```

#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

// Initialize OpenGL
void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-320, 320, -240, 240); // Coordinate system
}

// Draw X and Y axes
void drawAxes() {
    glColor3f(0.0, 0.0, 0.0); // Black axes
    glBegin(GL_LINES);
    glVertex2i(-320, 0); glVertex2i(320, 0); // X-axis
    glVertex2i(0, -240); glVertex2i(0, 240); // Y-axis
    glEnd();
}

// Bresenham's solid line
void drawBresenham(int x1, int y1, int x2, int y2) {
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);
    int sx = (x2 > x1) ? 1 : -1;
    int sy = (y2 > y1) ? 1 : -1;
    int err = dx - dy;
}

```

```

glColor3f(1.0, 0.0, 0.0); // Red line
glBegin(GL_POINTS);

while (true) {
    glVertex2i(x1, y1); // Plot point
    if (x1 == x2 && y1 == y2) break;
    int e2 = 2 * err;
    if (e2 > -dy) { err -= dy; x1 += sx; }
    if (e2 < dx) { err += dx; y1 += sy; }
}

glEnd();
}

// Display function
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    int x1, y1, x2, y2;
    cout << "Enter x1 y1 x2 y2: ";
    cin >> x1 >> y1 >> x2 >> y2;

    drawBresenham(x1, y1, x2, y2);
    glFlush();
}

// Main
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Solid Bresenham Line with Quadrants");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

4) Implement Bresenham circle drawing algorithm to draw any object. The object should be displayed in all the quadrants with respect to center and radius

```
#include <GL/glut.h>
```

```

#include <iostream>
#include <cmath>

// Draw a pixel at symmetric 8 points around the circle
void plotCirclePoints(int xc, int yc, int x, int y) {
    glBegin(GL_POINTS);
    glVertex2i(xc + x, yc + y);
    glVertex2i(xc - x, yc + y);
    glVertex2i(xc + x, yc - y);
    glVertex2i(xc - x, yc - y);
    glVertex2i(xc + y, yc + x);
    glVertex2i(xc - y, yc + x);
    glVertex2i(xc + y, yc - x);
    glVertex2i(xc - y, yc - x);
    glEnd();
}

// Bresenham's Circle Drawing Algorithm
void drawCircle(int xc, int yc, int r) {
    int x = 0, y = r;
    int d = 3 - 2 * r;

    while (x <= y) {
        plotCirclePoints(xc, yc, x, y);
        if (d < 0)
            d = d + 4 * x + 6;
        else {
            d = d + 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}

// Draw coordinate axes
void drawAxes() {
    glColor3f(1.0, 0.0, 0.0); // X-axis in red
    for (int i = -500; i <= 500; i++) {
        glBegin(GL_POINTS);
        glVertex2i(i, 0);
        glEnd();
    }

    glColor3f(0.0, 1.0, 0.0); // Y-axis in green
    for (int i = -500; i <= 500; i++) {
        glBegin(GL_POINTS);
        glVertex2i(0, i);
        glEnd();
    }
}

```

```

// Draw a flower-like object using multiple circles
void drawObject(int xc, int yc, int r) {
    glColor3f(0.0, 0.0, 1.0); // Blue circles

    drawCircle(xc, yc, r);      // Center circle
    drawCircle(xc + r, yc, r);  // Right petal
    drawCircle(xc - r, yc, r);  // Left petal
    drawCircle(xc, yc + r, r);  // Top petal
    drawCircle(xc, yc - r, r);  // Bottom petal
}

// Display callback
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();

    int radius = 40;

    // Draw object in all 4 quadrants
    drawObject( 100, 100, radius); // Quadrant I
    drawObject(-100, 100, radius); // Quadrant II
    drawObject(-100, -100, radius); // Quadrant III
    drawObject( 100, -100, radius); // Quadrant IV

    glFlush();
}

// Initialization
void init() {
    glClearColor(1, 1, 1, 1); // White background
    glColor3f(0, 0, 0);      // Default drawing color
    glPointSize(2.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-500, 500, -500, 500); // Center at (0, 0)
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 800);
    glutCreateWindow("Bresenham Circle Drawing - Object in All Quadrants");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

5) Implement the following polygon filling methods Flood fill / Seed fill using mouse click, keyboard interface and menu driven programming

```
#include <GL/glut.h>
#include <iostream>
using namespace std;

void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // White background
    glPointSize(1.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 640, 0, 480);
}

// Flood fill algorithm (4-directional)
void floodFill(int x, int y, float* fillColor, float* boundaryColor) {
    float oldColor[3];
    glReadPixels(x, y, 1, 1, GL_RGB, GL_FLOAT, oldColor);

    // Stop if pixel is the same as boundary color or already filled
    if ((oldColor[0] == boundaryColor[0] && oldColor[1] == boundaryColor[1] &&
        oldColor[2] == boundaryColor[2]) ||
        (oldColor[0] == fillColor[0] && oldColor[1] == fillColor[1] && oldColor[2] ==
        fillColor[2])) {
        return;
    }

    glColor3fv(fillColor);
    glBegin(GL_POINTS);
    glVertex2i(x, y);
    glEnd();
    glFlush();

    // Recursive flood fill in 4 directions
    floodFill(x + 1, y, fillColor, boundaryColor); // Right
    floodFill(x - 1, y, fillColor, boundaryColor); // Left
    floodFill(x, y + 1, fillColor, boundaryColor); // Up
    floodFill(x, y - 1, fillColor, boundaryColor); // Down
}

void draw_triangle() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw the triangle boundary
    glColor3f(0, 0, 0); // Black boundary
    glBegin(GL_LINE_LOOP);
        glVertex2i(300, 100);
        glVertex2i(300, 300);
        glVertex2i(450, 100);
    glEnd();
}
```

```

    glFlush();
}

void mouse(int button, int state, int x, int y) {
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        y = 480 - y; // Convert to OpenGL coordinates (Y is flipped)
        float boundaryColor[] = {0.0, 0.0, 0.0}; // Boundary is black
        float fillColor[] = {0.0, 1.0, 0.0}; // Filling color is green

        // Start flood fill inside the triangle at clicked position
        floodFill(x, y, fillColor, boundaryColor);
    }
}

void display() {
    draw_triangle();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(200, 200);
    glutCreateWindow("Flood Fill Inside Triangle");
    init();
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutMainLoop();
    return 0;
}

```

6)Implement the following polygon filling methods Boundary fill using mouse click, keyboard interface and menu driven programming

```

#include <iostream>
#include <math.h>
#include <time.h>
#include <GL/glut.h>

using namespace std;

void init(){
    glClearColor(1.0,1.0,1.0,0.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0,640,0,480);
}

void bound_it(int x, int y, float* fillColor, float* bc){

```

```

float color[3];
glReadPixels(x,y,1.0,1.0,GL_RGB,GL_FLOAT,color);
if((color[0]!=bc[0] || color[1]!=bc[1] || color[2]!=bc[2])&&(
color[0]!=fillColor[0] || color[1]!=fillColor[1] || color[2]!=fillColor[2])){
    glColor3f(fillColor[0],fillColor[1],fillColor[2]);
    glBegin(GL_POINTS);
        glVertex2i(x,y);
    glEnd();
    glFlush();
    bound_it(x+2,y,fillColor,bc);
    bound_it(x-2,y,fillColor,bc);
    bound_it(x,y+2,fillColor,bc);
    bound_it(x,y-2,fillColor,bc);
}
}

```

```

void mouse(int btn, int state, int x, int y){
    float bCol[] = {0,0,1};
    float color[] = {0,0,1};
    bound_it(x,y,color,bCol);
}

```

```

void world(){
    glLineWidth(3);
    glPointSize(2);
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0,0,1);
    glBegin(GL_LINE_LOOP);
        glVertex2i(150,100);
        glVertex2i(300,300);
        glVertex2i(450,100);
    glEnd();
    glFlush();
}

```

```

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(640,480);
    glutInitWindowPosition(200,200);
    glutCreateWindow("Many Amaze Very GL WOW");
    init();
    glutDisplayFunc(world);
    glutMouseFunc(mouse);
    glutMainLoop();
    return 0;
}

```

7) Implement Cohen Sutherland polygon clipping method to clip the polygon with respect the viewport and window. Use mouse click, keyboard interface

```
#include <GL/glut.h>
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

int xmin = 100, ymin = 100, xmax = 400, ymax = 300;
vector<pair<int, int>> polyPoints;
bool finishPoly = false;

// Region codes
const int INSIDE = 0; // 0000
const int LEFT = 1; // 0001
const int RIGHT = 2; // 0010
const int BOTTOM = 4; // 0100
const int TOP = 8; // 1000

int getCode(int x, int y) {
    int code = INSIDE;
    if (x < xmin) code |= LEFT;
    else if (x > xmax) code |= RIGHT;
    if (y < ymin) code |= BOTTOM;
    else if (y > ymax) code |= TOP;
    return code;
}

void clipLine(int x1, int y1, int x2, int y2, vector<pair<int, int>>& result) {
    int code1 = getCode(x1, y1);
    int code2 = getCode(x2, y2);
    bool accept = false;

    while (true) {
        if (!(code1 | code2)) { // Both inside
            accept = true;
            break;
        } else if (code1 & code2) { // Both outside same region
            break;
        } else {
            int code = code1 ? code1 : code2;
            int x, y;

            if (code & TOP) {
                x = x1 + (x2 - x1) * (ymax - y1) / (y2 - y1);
```

```

        y = ymax;
    } else if (code & BOTTOM) {
        x = x1 + (x2 - x1) * (ymin - y1) / (y2 - y1);
        y = ymin;
    } else if (code & RIGHT) {
        y = y1 + (y2 - y1) * (xmax - x1) / (x2 - x1);
        x = xmax;
    } else if (code & LEFT) {
        y = y1 + (y2 - y1) * (xmin - x1) / (x2 - x1);
        x = xmin;
    }

    if (code == code1) {
        x1 = x; y1 = y;
        code1 = getCode(x1, y1);
    } else {
        x2 = x; y2 = y;
        code2 = getCode(x2, y2);
    }
}

if (accept) {
    result.push_back({x1, y1});
    result.push_back({x2, y2});
}

}

void clipPolygon() {
    if (polyPoints.size() < 3) return;

    glClear(GL_COLOR_BUFFER_BIT);

    // Draw window
    glColor3f(0, 0, 0);
    glBegin(GL_LINE_LOOP);
    glVertex2i(xmin, ymin);
    glVertex2i(xmax, ymin);
    glVertex2i(xmax, ymax);
    glVertex2i(xmin, ymax);
    glEnd();

    // Draw original polygon in blue
    glColor3f(0, 0, 1);
    glBegin(GL_LINE_LOOP);
    for (auto& p : polyPoints) {
        glVertex2i(p.first, p.second);
    }
}

```

```

    }
    glEnd();

    // Clip and draw clipped polygon in red
    vector<pair<int, int>> clippedPoints;
    for (size_t i = 0; i < polyPoints.size(); i++) {
        int x1 = polyPoints[i].first;
        int y1 = polyPoints[i].second;
        int x2 = polyPoints[(i + 1) % polyPoints.size()].first;
        int y2 = polyPoints[(i + 1) % polyPoints.size()].second;

        clipLine(x1, y1, x2, y2, clippedPoints);
    }

    if (!clippedPoints.empty()) {
        glColor3f(1, 0, 0);
        glBegin(GL_LINES);
        for (size_t i = 0; i < clippedPoints.size(); i += 2) {
            glVertex2i(clippedPoints[i].first, clippedPoints[i].second);
            glVertex2i(clippedPoints[i+1].first, clippedPoints[i+1].second);
        }
        glEnd();
    }

    glFlush();
}

void display() {
    glClearColor(1, 1, 1, 0);
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw window
    glColor3f(0, 0, 0);
    glBegin(GL_LINE_LOOP);
    glVertex2i(xmin, ymin);
    glVertex2i(xmax, ymin);
    glVertex2i(xmax, ymax);
    glVertex2i(xmin, ymax);
    glEnd();

    // Draw current polygon points
    if (!polyPoints.empty()) {
        glColor3f(0, 0, 1);
        glBegin(GL_LINE_STRIP);
        for (auto& p : polyPoints) {
            glVertex2i(p.first, p.second);
        }
    }
}

```

```

    glEnd();

    glPointSize(5);
    glBegin(GL_POINTS);
    for (auto& p : polyPoints) {
        glVertex2i(p.first, p.second);
    }
    glEnd();
}

glFlush();
}

void mouse(int button, int state, int x, int y) {
    y = 480 - y; // Invert y coordinate

    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {
        if (!finishPoly) {
            polyPoints.push_back({x, y});
            display();
        }
    } else if (button == GLUT_RIGHT_BUTTON && state == GLUT_DOWN) {
        finishPoly = true;
        clipPolygon();
    }
}

void keyboard(unsigned char key, int x, int y) {
    if (key == 'c' || key == 'C') {
        polyPoints.clear();
        finishPoly = false;
        glClear(GL_COLOR_BUFFER_BIT);
        display();
    } else if (key == 'f' || key == 'F') {
        finishPoly = true;
        clipPolygon();
    } else if (key == 27) { // ESC
        exit(0);
    }
}

void init() {
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0, 640, 0, 480);
}

```

```

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutCreateWindow("Cohen-Sutherland Polygon Clipping");

    init();
    glutDisplayFunc(display);
    glutMouseFunc(mouse);
    glutKeyboardFunc(keyboard);

    glutMainLoop();
    return 0;
}

```

8) Implement Cohen Sutherland Line clipping method to clip the line with respect the viewport and window. Use mouse click, keyboard interface

```

#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

// Window coordinates
int x1, y_start, x2, y_end; // Line endpoints
int xmin = -100, ymin = -100, xmax = 100, ymax = 100; // Clipping window

// Region codes
const int INSIDE = 0; // 0000
const int LEFT = 1; // 0001
const int RIGHT = 2; // 0010
const int BOTTOM = 4; // 0100
const int TOP = 8; // 1000

int computeCode(int x, int y) {
    int code = INSIDE;

    if (x < xmin)
        code |= LEFT;
    else if (x > xmax)
        code |= RIGHT;

    if (y < ymin)
        code |= BOTTOM;
    else if (y > ymax)
        code |= TOP;

    return code;
}

```

```

void cohenSutherlandClip() {
    int code1 = computeCode(x1, y_start);
    int code2 = computeCode(x2, y_end);
    bool accept = false;

    while (true) {
        if (!(code1 | code2)) { // Both endpoints inside window
            accept = true;
            break;
        }
        else if (code1 & code2) { // Both endpoints outside window
            break;
        }
        else {
            int code_out = code1 ? code1 : code2;
            int x, y;

            // Find intersection point
            if (code_out & TOP) {
                 $x = x1 + (x2 - x1) * (ymax - y_start) / (y_end - y_start);$ 
                y = ymax;
            }
            else if (code_out & BOTTOM) {
                 $x = x1 + (x2 - x1) * (ymin - y_start) / (y_end - y_start);$ 
                y = ymin;
            }
            else if (code_out & RIGHT) {
                 $y = y_start + (y_end - y_start) * (xmax - x1) / (x2 - x1);$ 
                x = xmax;
            }
            else if (code_out & LEFT) {
                 $y = y_start + (y_end - y_start) * (xmin - x1) / (x2 - x1);$ 
                x = xmin;
            }
        }

        // Replace outside point with intersection point
        if (code_out == code1) {
            x1 = x;
            y_start = y;
            code1 = computeCode(x1, y_start);
        }
        else {
            x2 = x;
            y_end = y;
            code2 = computeCode(x2, y_end);
        }
    }
}

```

```

    glColor3f(1.0, 1.0, 1.0);
    glBegin(GL_LINE_LOOP); // Draw clipping window
    glVertex2i(xmin, ymin);
    glVertex2i(xmax, ymin);
    glVertex2i(xmax, ymax);
    glVertex2i(xmin, ymax);
    glEnd();

    if (accept) {
        glColor3f(1.0, 0.0, 0.0);
        glBegin(GL_LINES);
        glVertex2i(x1, y_start);
        glVertex2i(x2, y_end);
        glEnd();
    }

    glFlush();
}

void init() {
    glClearColor(0.0, 0.0, 0.0, 0.0);
    gluOrtho2D(-320, 320, -240, 240);
    glClear(GL_COLOR_BUFFER_BIT);
}

int main(int argc, char** argv) {
    cout << "Enter line coordinates (x1 y_start x2 y_end): ";
    cin >> x1 >> y_start >> x2 >> y_end;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowPosition(100, 100);
    glutInitWindowSize(640, 480);
    glutCreateWindow("Cohen-Sutherland Line Clipping");
    init();
    glutDisplayFunc(cohenSutherlandClip);
    glutMainLoop();
    return 0;
}

```

9) Implement DDA line drawing algorithm to draw Staircase line. Divide the screen in four quadrants with center as (0, 0). The line should work for all the slopes

```

#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

void init() {

```

```

    glClearColor(1, 1, 1, 1);          // White background
    gluOrtho2D(-320, 320, -240, 240);  // Set coordinate system
}

void drawAxes() {
    glColor3f(0, 0, 0); // Black
    glBegin(GL_LINES);
    glVertex2i(-320, 0);
    glVertex2i(320, 0);
    glVertex2i(0, -240);
    glVertex2i(0, 240);
    glEnd();
}

// DDA line drawing function
void drawDDA(int x1, int y1, int x2, int y2) {
    float dx = x2 - x1;
    float dy = y2 - y1;
    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);
    float xInc = dx / steps;
    float yInc = dy / steps;
    float x = x1, y = y1;

    glBegin(GL_POINTS);
    for (int i = 0; i <= steps; ++i) {
        glVertex2i(round(x), round(y));
        x += xInc;
        y += yInc;
    }
    glEnd();
}

// Draws a staircase starting from origin (0,0)
void drawStaircaseFromCenter(int steps, int size) {
    glColor3f(1, 0, 0); // Red

    int x = 0, y = 0;
    for (int i = 0; i < steps; i++) {
        drawDDA(x, y, x + size, y); // Horizontal
        x += size;
        drawDDA(x, y, x, y + size); // Vertical
        y += size;
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawAxes();
    drawStaircaseFromCenter(5, 30); // 5 steps of size 30
    glFlush();
}

```

```

}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Centered DDA Staircase with Axes");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```

10) Implement following 2D transformations on the object with respect to axis

a. Scaling

b. Rotation about arbitrary point

c. Reflection

```

#include <GL/glut.h>
#include <iostream>
#include <cmath>

float angle = 0; // Rotation angle
float scaleX = 1, scaleY = 1; // Scaling factors
float reflectX = 1, reflectY = 1; // Reflection factors

float centerX = 200, centerY = 200; // Arbitrary point for rotation

// Coordinates of the object (e.g., a square)
float obj[4][2] = {
    {100, 100}, {300, 100}, {300, 300}, {100, 300} // Square vertices
};

// Function to draw the object
void drawObject() {
    glBegin(GL_POLYGON);
    for (int i = 0; i < 4; i++) {
        glVertex2f(obj[i][0], obj[i][1]);
    }
    glEnd();
}

// Function to apply scaling transformation
void scaleObject() {
    for (int i = 0; i < 4; i++) {
        obj[i][0] *= scaleX; // Scale X coordinate
        obj[i][1] *= scaleY; // Scale Y coordinate
    }
}

```

```

}

// Function to apply rotation about an arbitrary point
void rotateObject() {
    float tempX, tempY;
    float rad = angle * M_PI / 180.0f; // Convert angle to radians
    float cosA = cos(rad);
    float sinA = sin(rad);

    for (int i = 0; i < 4; i++) {
        // Translate object to origin
        tempX = obj[i][0] - centerX;
        tempY = obj[i][1] - centerY;

        // Apply rotation
        obj[i][0] = centerX + (tempX * cosA - tempY * sinA);
        obj[i][1] = centerY + (tempX * sinA + tempY * cosA);
    }
}

// Function to apply reflection over X and Y axes
void reflectObject() {
    for (int i = 0; i < 4; i++) {
        obj[i][0] *= reflectX; // Reflect X coordinate
        obj[i][1] *= reflectY; // Reflect Y coordinate
    }
}

// Function to handle keyboard inputs for transformations
void keyboard(unsigned char key, int x, int y) {
    if (key == 's') {
        scaleX = 1.5; scaleY = 1.5; // Scale by a factor of 1.5
        scaleObject();
    } else if (key == 'r') {
        angle += 10; // Rotate by 10 degrees
        rotateObject();
    } else if (key == 'f') {
        reflectX = -1; // Reflect over the X-axis
        reflectY = 1;
        reflectObject();
    } else if (key == 'F') {
        reflectX = 1; // Reset reflection to no reflection
        reflectY = 1;
        reflectObject();
    } else if (key == 'q') {
        exit(0); // Quit on 'q'
    }
    glutPostRedisplay(); // Refresh the display
}

```

```

// Function to display the object and apply transformations
void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0, 1.0, 0.0); // Green color for object
    drawObject(); // Draw the transformed object

    glFlush();
}

// Initialization function
void init() {
    glClearColor(1.0, 1.0, 1.0, 1.0); // Set background color to white
    glColor3f(0.0, 0.0, 0.0); // Set drawing color to black
    glPointSize(2);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 800, 0, 800); // Set up the projection matrix (2D)
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(800, 800);
    glutCreateWindow("2D Transformations");

    init();
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);

    glutMainLoop();
    return 0;
}

```

11)Generate fractal patterns using Bezier Curve

```

#include <GL/glut.h>
#include <iostream>
#include <vector>
#include <cmath>
using namespace std;

struct Point {
    float x, y;
};

vector<Point> ctrlPoints;
int numCtrlPoints;

void init() {

```

```

    glClearColor(1, 1, 1, 1); // White background
    glColor3f(0, 0, 1);      // Blue curve
    glPointSize(5.0);        // Bigger points
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 640, 0, 480);
}

int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int binomial(int n, int k) {
    return factorial(n) / (factorial(k) * factorial(n - k));
}

Point bezierPoint(const vector<Point>& points, float t) {
    int n = points.size() - 1;
    Point p = {0, 0};
    for (int i = 0; i <= n; ++i) {
        float coeff = binomial(n, i) * pow(t, i) * pow(1 - t, n - i);
        p.x += points[i].x * coeff;
        p.y += points[i].y * coeff;
    }
    return p;
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw control points
    glColor3f(1, 0, 0); // Red control points
    glBegin(GL_POINTS);
    for (auto& p : ctrlPoints) {
        glVertex2f(p.x, p.y);
    }
    glEnd();

    // Draw Bezier curve
    glColor3f(0, 0, 1); // Blue curve
    glBegin(GL_LINE_STRIP);
    for (float t = 0; t <= 1.0; t += 0.01) {
        Point p = bezierPoint(ctrlPoints, t);
        glVertex2f(p.x, p.y);
    }
    glEnd();

    glFlush();
}

```

```

int main(int argc, char** argv) {
    cout << "Enter number of control points: ";
    cin >> numCtrlPoints;

    ctrlPoints.resize(numCtrlPoints);
    for (int i = 0; i < numCtrlPoints; i++) {
        cout << "Enter control point " << i + 1 << " (x y): ";
        cin >> ctrlPoints[i].x >> ctrlPoints[i].y;
    }

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Simple Bezier Curve");
    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```

12)Generate fractal patterns using Koch Curve.

```

#include <iostream>
#include <cmath>
#include <GL/glut.h>
#define M_PI 3.14159265358979323846

using namespace std;

// Global variables for input
double x, y, len, angle;
int it;

// Initialize OpenGL settings
void init() {
    glClearColor(1.0, 1.0, 1.0, 0.0); // White background
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, 640, 0, 480); // 2D coordinate system
}

// Function to draw a single line
void drawLine(double x1, double y1, double x2, double y2) {
    glBegin(GL_LINES);
    glVertex2d(x1, y1);
    glVertex2d(x2, y2);
    glEnd();
}

// Recursive Koch Curve function

```

```

void koch(double x, double y, double len, double angle, int it) {
    if (it == 0) {
        double nx = x + len * cos(angle * M_PI / 180);
        double ny = y + len * sin(angle * M_PI / 180);
        drawLine(x, y, nx, ny);
        return;
    }

    len /= 3;
    koch(x, y, len, angle, it - 1);

    x += len * cos(angle * M_PI / 180);
    y += len * sin(angle * M_PI / 180);
    koch(x, y, len, angle + 60, it - 1);

    x += len * cos((angle + 60) * M_PI / 180);
    y += len * sin((angle + 60) * M_PI / 180);
    koch(x, y, len, angle - 60, it - 1);

    x += len * cos((angle - 60) * M_PI / 180);
    y += len * sin((angle - 60) * M_PI / 180);
    koch(x, y, len, angle, it - 1);
}

// Display callback
void display() {
    glClear(GL_COLOR_BUFFER_BIT); // Clear screen
    glColor3f(0, 0, 0);          // Set drawing color to black
    koch(x, y, len, angle, it);   // Draw Koch curve
    glFlush();                    // Show the result
}

int main(int argc, char** argv) {
    // Get user input
    cout << "Enter starting point (x y): ";
    cin >> x >> y;
    cout << "Enter length and angle: ";
    cin >> len >> angle;
    cout << "Enter number of iterations: ";
    cin >> it;

    // Initialize GLUT
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(640, 480);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Koch Curve");

    init(); // Call initialization
    glutDisplayFunc(display); // Set display function
}

```

```

    glutMainLoop();           // Start main loop
    return 0;
}

```

13 Implement animation principles for any object

```

#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

float x = 100, y = 100; // Position of object
float dx = 2, dy = 1;   // Speed of movement
float obj_size = 30;    // Size of object (renamed from size to obj_size)
int windowWidth = 640, windowHeight = 480;

void init() {
    glClearColor(1, 1, 1, 0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0, windowWidth, 0, windowHeight);
}

void drawObject() {
    // Draw a simple circle
    glColor3f(1, 0, 0);
    glBegin(GL_POLYGON);
    for (int i = 0; i < 360; i += 30) {
        float rad = i * M_PI / 180;
        glVertex2f(x + obj_size * std::cos(rad), y + obj_size * std::sin(rad));
    }
    glEnd();
    glFlush();
}

void animate(int value) {
    // Clear the screen
    glClear(GL_COLOR_BUFFER_BIT);

    // Update position with bouncing
    x += dx;
    y += dy;

    // Bounce off walls
    if (x > windowWidth - obj_size || x < obj_size) dx = -dx;
    if (y > windowHeight - obj_size || y < obj_size) dy = -dy;

    // Draw the object
    drawObject();
}

```

```

    // Call animate again after 16ms (about 60fps)
    glutTimerFunc(16, animate, 0);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    drawObject();
}

int main(int argc, char** argv) {
    cout << "Initial position (x y): ";
    cin >> x >> y;
    cout << "Speed (dx dy): ";
    cin >> dx >> dy;
    cout << "Size: ";
    cin >> obj_size;

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(windowWidth, windowHeight);
    glutInitWindowPosition(200, 200);
    glutCreateWindow("Animation");

    init();
    glutDisplayFunc(display);
    glutTimerFunc(0, animate, 0);
    glutMainLoop();

    return 0;
}

```

Compile and Run commands:

Compile: g++ filename.cpp -o filename -lGL -lGLU -lglut

Run: ./filename